Milestone 1 — Réparation et mise en ordre du projet

> Créé le 1er avril 2026 — Phases 1 à 4 du Planning MVP **Objectif** : Remettre le projet sur de bonnes bases avant d'ajouter de nouvelles fonctionnalités.

Pourquoi ?

Le projet fonctionne partiellement, mais :

- **Les données ne sont pas protégées** : tout utilisateur connecté peut voir les données de tout le monde
- **Le code est désorganisé** : certains fichiers font plus de 1 500 lignes
- **Aucun filet de sécurité** : 0 test automatique, 0 vérification avant publication
- **Aucune règle d'équipe** : 0 revue de code, 0 suivi de bugs, tout le monde pousse directement sur le code principal

Ce milestone ne change rien de visible pour l'utilisateur, mais sans lui on ne peut pas construire la suite.

Vue d'ensemble

Étape	Nom	Résumé
1	Sécurité et base de données	Sécuriser les données, réparer les fondations
2	Organisation du travail d'équipe	Règles de collaboration sur le code
3	Tests automatiques	Vérification automatique à chaque modification
4	Rangement du code	Réorganisation pour la lisibilité et la maintenabilité

Étape 1 — Sécurité et base de données

> **Priorité MAXIMALE — Tout le reste en dépend.**

Vocabulaire utile

Terme	Définition
Base de données	L'endroit où toutes les informations sont stockées (familles, enfants, centres...). On utilise **Supabase**, un service en ligne.
Table	Un tableau dans la base de données. Chaque table stocke un type de données (ex : une table « familles », une table « enfants »).
RLS (Row Level Security)	Règle qui dit : « Tu ne peux voir que tes propres données ». Ex : la famille Dupont ne voit pas les enfants de la famille Martin.

Terme	Définition
Index	Comme l'index d'un livre : permet à la base de données de trouver une information sans tout parcourir.
Middleware	Un gardien placé entre l'utilisateur et le site, qui vérifie les droits d'accès.
Migration	Un fichier numéroté qui décrit un changement de structure de la base de données. Permet de suivre l'historique et de revenir en arrière.
Validation	Vérifier que les données reçues respectent le format attendu (un email contient un « @ », un téléphone contient 10 chiffres…).
Rate limiting	Limiteur de vitesse : « Maximum 5 demandes par minute, au-delà tu es bloqué temporairement. »
Error Boundary	Filet de sécurité : si un morceau de page plante, seul ce morceau affiche une erreur, le reste de la page continue de fonctionner.

1.1 — Réparer la porte d'entrée du site

La réinitialisation du mot de passe permet de changer le mot de passe de n'importe qui sans vérification par email. De plus, une liste d'emails autorisés existe en base mais le code ne la consulte jamais lors de l'inscription.

→ Sécuriser le reset mot de passe (confirmation par email) et vérifier les emails autorisés à l'inscription.

1.2 — Supprimer une faille de sécurité critique

La « clé maître » qui donne un accès total à la base de données pourrait se retrouver visible dans le navigateur à cause d'un mécanisme de secours dans le code.

→ Supprimer ce mécanisme de secours.

1.3 — Créer les 3 tables manquantes

Le code fait référence à 3 tables qui n'existent pas, ce qui provoque des plantages :

- **inscriptions** (inscriptions aux séjours)
- **projects** (projets pédagogiques)
- **project_edit_log** (historique des modifications)

1.4 — Corriger les types de données des enfants

L'identifiant et la date de naissance des enfants sont stockés en texte libre au lieu de formats adaptés. C'est comme écrire « douze mars deux mille quinze » au lieu de « 12/03/2015 » dans un tableur — impossible de faire des calculs.

→ Identifiant en numéro automatique, date de naissance en vrai format date.

1.5 — Protéger les données de chaque utilisateur (RLS)

Sur 9 tables, **une seule** a des règles de protection. Les 8 autres sont en accès libre pour tout utilisateur connecté = violation RGPD.

→ Ajouter les règles suivantes :

- **Familles** : chaque utilisateur ne voit que les siennes
- **Adultes et enfants** : accessibles seulement via la famille liée
- **Centres et partenaires** : lecture pour tous, écriture pour les admins
- **Emails autorisés** : accès système uniquement

1.6 — Ajouter des index

Ajouter des index sur les colonnes les plus recherchées (email adulte, lien enfant↔famille, lien contact↔centre) pour accélérer les requêtes.

1.7 — Ajouter des en-têtes de sécurité

La configuration du site est quasiment vide. Il manque les instructions de sécurité standard envoyées au navigateur (empêcher l'intégration dans un autre site, bloquer les scripts non autorisés…).

1.8 — Limiter les abus sur les formulaires publics

Les formulaires d'inscription et de mot de passe oublié ne sont pas protégés : un robot pourrait envoyer des milliers de demandes par seconde.

→ Mettre en place un limiteur de vitesse (rate limiting).

1.9 — Vérifier les données envoyées par les utilisateurs

Le logiciel ne vérifie pas côté serveur si les données sont correctes. L'outil **Zod** est déjà installé mais jamais utilisé.

→ Créer des vérifications pour chaque formulaire et chaque route.

1.10 — Masquer les erreurs techniques

Les messages d'erreur bruts de la base de données sont renvoyés tels quels à l'utilisateur, révélant la structure interne.

→ Messages simples côté utilisateur, détails techniques dans les journaux côté serveur.

1.11 — Mettre en place les rôles utilisateur

Le logiciel connaît 3 rôles (administrateur, encadrant, direction) mais ne les utilise pas. Tout le monde a accès à tout.

→ Stocker les rôles en base, vérifier le rôle via le middleware, restreindre les pages sensibles.

1.12 — Gestion des erreurs d'affichage

Si un morceau de page plante, toute la page devient blanche.

→ Ajouter des Error Boundaries aux endroits critiques.

1.13 — Mettre en place les migrations

Sauvegarder l'état actuel de la base de données et versionner tous les changements futurs via Supabase CLI.

Récapitulatif Étape 1

Tâche	Priorité
Réparer la porte d'entrée (middleware + signup)	Critique
Supprimer la faille de la clé maître	Critique
Créer les 3 tables manquantes	Critique
Protéger les données (RLS) sur 8 tables	Critique
Corriger les types de données des enfants	Élevée
Rôles utilisateur + contrôle d'accès	Élevée
Migrations de base de données	Élevée
Ajouter les index	Moyenne
En-têtes de sécurité	Moyenne
Rate limiting	Moyenne
Validation des données (Zod)	Moyenne
Masquer les erreurs techniques	Moyenne
Error boundaries	Moyenne

Étape 2 — Organisation du travail d'équipe

> **Priorité CRITIQUE — Sans règles, chaque modification du code est un risque.**

Vocabulaire utile

Terme	Définition
Commit	Une « sauvegarde » du code à un instant donné, avec un message qui décrit ce qui a changé.
Branche	Une copie temporaire du projet pour travailler sans risquer de casser le code principal.
PR (Pull Request)	Demande de relecture : « J'ai terminé, quelqu'un peut vérifier avant qu'on intègre ? »
Git flow	Le circuit de travail qui organise les branches et les fusions.

La situation actuelle

Élément	État
Branches	**1 seule** — tout le monde travaille au même endroit
Revue de code	**0** — aucune relecture
Suivi des bugs	**0 ticket** — bugs dans un fichier texte local
Versions	**0** — aucun point de repère
Format des commits	Aucune convention

2.1 — Définir un circuit de travail

Exemple, trois branches principales :

1. `main` = le code en production
2. `develop` = la zone d'assemblage avant production
3. `prenom/…` = une branche par développeur (ex : `adele/corrections-ui`, `jeanne/page-direction`)

Chaque dev travaille sur sa branche, puis fusionne dans `develop` via une PR. Quand `develop` est stable, on fusionne dans `main`.

Circuit : `Travailler sur sa branche → PR vers develop → Relecture → Fusionner`

2.2 — Protéger le code principal

Configurer GitHub pour interdire les modifications directes sur `main`. Toute modification devra passer par une PR, être approuvée par au moins une personne, et réussir les vérifications automatiques.

2.3 — Standardiser les messages de commit

Adopter le format *Conventional Commits* :

Préfixe	Signification	Exemple
`feat:`	Nouvelle fonctionnalité	`feat: ajouter le formulaire inscription séjour`
`fix:`	Correction de bug	`fix: corriger le calcul de l'âge`
`refactor:`	Réorganisation sans changer le comportement	`refactor: découper la page clients`
`docs:`	Documentation	`docs: mettre à jour le guide d'installation`
`test:`	Tests	`test: ajouter les tests du formulaire famille`

Un outil refusera les messages non conformes.

2.4 — Créer un système de suivi des bugs

Les ~30 bugs du fichier `bugfix.txt` seront convertis en tickets GitHub avec des modèles (bug, fonctionnalité, refactoring), assignés et priorisés.

2.5 — Marquer le point de départ

Créer un tag de version `v0.1.0` sur l'état actuel — un marque-page pour pouvoir revenir en arrière si besoin.

2.6 — Écrire un guide pour l'équipe

Un `CONTRIBUTING.md` qui explique comment créer une branche, nommer ses commits, faire une PR, et ce qu'on attend d'une relecture.

Récapitulatif Étape 2

Tâche	Priorité
Définir le circuit de travail (Git flow)	Critique
Protéger le code principal	Critique
Standardiser les messages de commit	Élevée
Convertir bugfix.txt en tickets GitHub	Élevée
Tag v0.1.0	Moyenne
Guide de contribution	Moyenne

Étape 3 — Tests automatiques

> **Priorité CRITIQUE — 0 test aujourd'hui. Chaque modification peut casser quelque chose sans qu'on s'en rende compte.**

Vocabulaire utile

Terme	Définition
Test unitaire	Vérifie un petit morceau de code isolé (une fonction, un bouton).
Test d'intégration	Vérifie que plusieurs morceaux fonctionnent ensemble.
Test E2E (bout en bout)	Simule un vrai utilisateur dans un navigateur (il ouvre le site, clique, tape, vérifie).
Pipeline CI	Un robot GitHub qui vérifie automatiquement le code, la construction du projet et les tests à chaque PR. Si ça échoue, la PR est bloquée.

3.1 — Installer les outils

- **Vitest** : exécute les tests unitaires et d'intégration
- **Testing Library** : teste l'interface comme un utilisateur (cliquer, remplir un formulaire…)
- **Playwright** : navigateur automatisé pour les tests E2E

3.2 — Créer le pipeline CI

Un robot GitHub Actions qui se déclenche à chaque PR et fait : vérification du code → construction du projet → lancement des tests. Si une étape échoue, impossible de fusionner.

3.3 — Écrire les premiers tests

On commence par le plus critique :

Logique métier : CRUD familles, calcul d'âge, formatage téléphone, détection doublons

Interface : formulaire de connexion, validation des champs, messages d'erreur

Routes serveur : refus email non autorisé, validation mot de passe, rejet utilisateurs non connectés

3.4 — Organiser les dossiers

```
__tests__/
├── unit/           ← Fonctions isolées (services, utilitaires, composants)
├── integration/    ← Routes API, communication base de données
└── e2e/            ← Parcours utilisateur complet
```

Récapitulatif Étape 3

Tâche	Priorité
Installer Vitest + Testing Library	Critique
Créer le pipeline CI	Critique
Tests des services (logique métier)	Critique
Tests des routes API	Élevée
Tests des composants clés	Élevée
Installer Playwright	Moyenne
Organiser les dossiers de tests	Moyenne

Étape 4 — Rangement du code

> **Priorité HAUTE — Code fonctionnel mais très difficile à faire évoluer.**

Vocabulaire utile

Terme	Définition
Composant	Un morceau d'interface réutilisable (bouton, formulaire, tableau…). Comme des briques LEGO qu'on assemble pour construire une page.
Service	Un fichier de logique pour un domaine précis (familles, centres…). Fait le lien entre l'interface et la base de données.
Type	Description de la forme d'une donnée : « Une famille a un identifiant (nombre), un nom (texte), un email (texte)… ».
Server / Client Component	Code qui s'exécute sur le serveur (prépare les données) vs dans le navigateur (interactivité). Tout mettre côté navigateur = site plus lent.
Groupe de routes	Dossier spécial dans Next.js. Toutes les pages d'un groupe partagent la même présentation (même menu, même barre latérale).

L'état des lieux

Problème	Actuel	Objectif
Fichiers à la racine	19 (docs + config + SQL mélangés)	~10 (config uniquement)
Plus gros fichier	`clients/page.tsx` = **1 500+ lignes**	~200 lignes max par fichier
Plus gros service	`services/api.ts` = **900+ lignes**	4-5 fichiers spécialisés
Styles CSS	3 méthodes mélangées	1 seule (Tailwind)
Pages en double	`familles/` + `clients/`, `inscription/` + `inscriptions/`	0 doublon

4A — Ranger la racine

Déplacer la documentation dans `docs/`, les migrations SQL dans `supabase/migrations/`, créer un `.env.example`, supprimer les fichiers et dossiers vides.

4B — Organiser les pages par groupe

- **`(auth)/`** : login, signup, reset-password → affichage simple, sans menu
- **`(app)/`** : toutes les pages du logiciel → avec navigation et barre latérale
- Supprimer les pages en double (`familles/` et `inscription/`)

4C — Découper les fichiers trop gros

Exemple — `clients/page.tsx` (1 500+ lignes, tout dans un seul fichier) :

```
Avant : 1 fichier qui gère tout (liste, formulaire, tableau enfants, fiche
sanitaire, filtres, doublons…)

Après : 1 page de 20 lignes + 5-6 composants spécialisés dans components/families/
```

Mêmes découpages pour `fiche/page.tsx`, `centres/[id]/page.tsx`, `homepage/page.tsx`, `sejours/page.tsx`.

4D — Optimiser serveur vs navigateur

~6 composants sont marqués « interactifs » alors qu'ils n'ont aucune interactivité (Button, IconButton, Modal…). Les retirer du navigateur et transformer les pages en composants serveur → site plus rapide.

4E — Découper les services

`services/api.ts` (900+ lignes) → `services/families.ts` + `services/adults.ts` + `lib/mappers.ts`. Créer aussi `services/centres.ts` et `services/sejours.ts` (squelettes).

4F — Réorganiser les composants

Déplacer les composants mal classés dans les bons dossiers. Créer les composants de base manquants (champ de saisie, menu déroulant, tableau de données, barre de recherche, boîte de confirmation).

4G — Unifier le CSS

3 méthodes mélangées actuellement (Tailwind, CSS Modules, @apply). Tout convertir en Tailwind, supprimer le reste.

4H — Centraliser les types

Les définitions de données sont dispersées partout. Les regrouper dans `types/` avec un fichier par domaine : `family.ts`, `centre.ts`, `inscription.ts`, `auth.ts`…

Récapitulatif Étape 4

Sous-étape	Action
4A. Ranger la racine	Déplacer les documents, créer la structure
4B. Grouper les pages	Séparer auth / appli, supprimer doublons
4C. Découper les fichiers	Fichiers géants → petits composants
4D. Serveur vs navigateur	Optimiser ce qui s'exécute où
4E. Découper les services	Logique métier → 1 fichier par domaine
4F. Réorganiser les composants	Ranger + créer les manquants
4G. Unifier le CSS	Tout en Tailwind

Sous-étape	Action
4H. Centraliser les types	1 source de vérité par domaine

Résumé

Étape	Thème	Apport
1	Sécurité et base de données	Données protégées, failles comblées, base solide
2	Organisation du travail d'équipe	Règles claires, code relu avant publication
3	Tests automatiques	Régressions détectées automatiquement
4	Rangement du code	Projet lisible, chaque fichier a un rôle clair

Pour l'utilisateur : rien de visible, mais ses données sont réellement protégées et le logiciel est plus rapide.

Pour l'équipe : circuit de validation, suivi des bugs, tests automatiques, code organisé, mêmes règles pour tous.

→ **Une fois ce milestone terminé, le projet est prêt pour les nouvelles fonctionnalités (séjours, partenaires, facturation, site vitrine…) sans risquer de tout casser.**